

Session 6 — Inter-Service Communication

Part 1 — Foundations of Communication in Microservices

Session Goal

By the end of this session, students should understand:

- Why microservices need communication
 - The difference between **Synchronous** and **Asynchronous** communication
 - Blocking vs Non-Blocking execution
 - Imperative vs Fluent vs Declarative vs Reactive programming
 - Java **CompletableFuture**
 - Reactive Programming
 - Mono & Flux
 - Eventual Consistency
 - How to choose the correct communication model before selecting a technology
-

Learning Objectives

After completing this lesson, students will be able to:

- ✓ Explain why microservices communicate over the network.
- ✓ Distinguish between synchronous and asynchronous communication.
- ✓ Explain blocking and non-blocking execution.
- ✓ Understand imperative, fluent, declarative, and reactive programming styles.
- ✓ Understand Java CompletableFuture.
- ✓ Explain Reactive Programming.
- ✓ Differentiate between Mono and Flux.
- ✓ Explain Eventual Consistency.

✓ Decide whether a business scenario should use synchronous or asynchronous communication.

1. Why Do Microservices Need Communication?

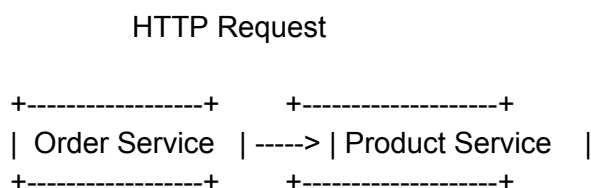
Unlike a monolithic application, each microservice is an independent application.

Each service:

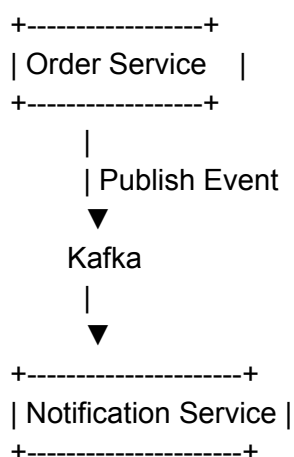
- has its own database
- has its own business logic
- runs independently
- can be deployed independently

Since services cannot directly call each other's Java methods, they must communicate over the network.

Example:



or



This communication is called:

Inter-Service Communication

Real Business Example

Imagine an e-commerce platform.

A customer clicks **Place Order**.

What happens?

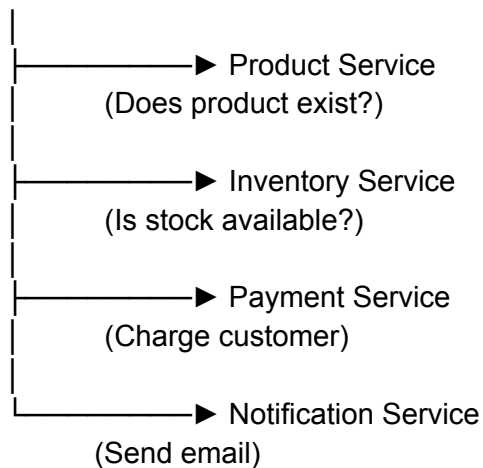
Client



Order Service

Order Service now needs information from other services.

Order Service



Notice that not all requests are equally important.

Some require an immediate answer.

Others do not.

The Most Important Question

Before choosing any technology, always ask:

Do I need the answer immediately?

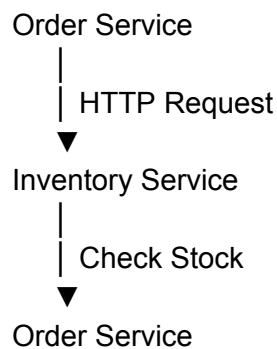
This single question determines almost everything.

2. Two Communication Models

Microservices communicate in only two ways.

1. Synchronous Communication

The caller sends a request **and waits** for the response.



Order Service cannot continue until Inventory replies.

Think of making a phone call.

You dial the number...

You wait...

The other person answers...

Only then can the conversation continue.

Characteristics of Synchronous Communication

- ✓ Immediate response
 - ✓ Easy to understand
 - ✓ Usually uses HTTP or gRPC
 - ✓ Usually Blocking
-

Business Example

Customer wants to place an order.

Before creating the order, we must know whether stock exists.

Order Service

↓

Inventory Service

↓

Stock = 20

↓

Create Order

Without knowing stock availability, the order cannot continue.

Therefore:

Synchronous communication is the correct choice.

Advantages

- Simple
 - Immediate answer
 - Easy debugging
 - Straightforward programming model
-

Disadvantages

Suppose Inventory Service becomes slow.

Order Service

↓

Waiting...

↓

Waiting...

↓

Waiting...

↓

Inventory Service

The Order Service cannot continue.

The thread remains occupied.

Performance decreases.

2. Asynchronous Communication

Instead of waiting,

the service sends a message

and immediately continues.

Order Service

↓

Publish Event

↓

Kafka

↓

Notification Service

Order Service never waits.

Think about sending an email.

You do not wait until someone reads it.

You simply send it and continue working.

Business Example

Customer places an order.

Immediately:

Create Order

↓

Return 201 Created

A few seconds later:

Notification Service

↓

Send Confirmation Email

The customer does not need to wait for the email.

Characteristics

- ✓ No waiting
 - ✓ Better scalability
 - ✓ Independent services
 - ✓ Better fault tolerance
-

Advantages

- High throughput
- Loose coupling
- Excellent scalability

Disadvantages

- More complex
 - Harder debugging
 - Response is not immediate
-

Synchronous vs Asynchronous

Feature	Synchronous	Asynchronous
Waits for response	Yes	No
Immediate result	Yes	No
Typical Protocol	HTTP, gRPC	Kafka, RabbitMQ
Complexity	Lower	Higher
Scalability	Moderate	Excellent
Typical Use	Read operations, validation	Notifications, analytics, logging

3. Decision Framework

Always ask these three questions.

Question 1

Do I need the answer immediately?

YES

↓

Use **Synchronous Communication**

NO

↓

Use **Asynchronous Communication**

Question 2

Can this operation happen later?

Example:

Sending an email.

YES

↓

Async

Question 3

Can I accept a short delay?

If yes,

Async is acceptable.

If no,

use Sync.

Business Mapping

Operation	Communication
Order → Inventory	Sync
Order → Product	Sync
Order → Payment	Sync
Order → Notification	Async

Order → Audit Log Async

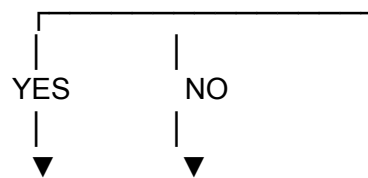
Order → Analytics Async

Final Decision Tree

Need communication?



Need the answer NOW?



Sync

Async

Everything else in this chapter builds on this decision.

4. Blocking vs Non-Blocking

Many beginners confuse these concepts.

Blocking is NOT the opposite of **Asynchronous**.

Blocking describes **what happens to the thread**.

Blocking Execution

The thread waits until the response arrives.

Request



Waiting...

↓

Waiting...

↓

Response

↓

Continue

During the waiting period,
the thread cannot perform any other work.

Example

```
Product product = productClient.getProduct(id);
```

```
System.out.println(product.getName());
```

The second line cannot execute until the first finishes.

Characteristics

- Simple programming
 - Easy debugging
 - One thread per request
-

Non-Blocking Execution

The thread sends the request and immediately becomes free.

Request

↓

Continue doing other work

↓

Response arrives



Handle response

The thread never sits idle.

Why is this important?

Imagine 10,000 users.

Blocking

10,000 Requests



10,000 Threads

Non-Blocking

10,000 Requests



20 Threads



Event Loop

Huge memory savings.

Better scalability.

Blocking vs Non-Blocking

Blocking

Thread waits

Simple

Non-Blocking

Thread continues

More complex

Lower scalability	Higher scalability
Traditional MVC	Reactive Systems

Important Note

Blocking and Non-Blocking are **execution models**.

They do NOT describe business communication.

For example:

- HTTP can be Blocking.
- HTTP can also be Non-Blocking.

It depends on the client library.

5. Programming Styles

Spring supports multiple programming styles.

Many students confuse them.

Imperative Programming

Traditional Java programming.

You explicitly describe each step.

```
Product product = service.getProduct(id);
```

```
System.out.println(product.getName());
```

Execution is line by line.

Characteristics

- ✓ Simple
 - ✓ Easy to read
 - ✓ Traditional Java
-

Fluent API

Instead of multiple statements,
methods are chained together.

Example:

```
webClient.get()  
    .uri("/products/1")  
    .retrieve()  
    .bodyToMono(Product.class);
```

Each method returns another object,
allowing readable chains.

Characteristics

- More readable
 - Method chaining
 - Less boilerplate
-

Declarative Programming

Instead of telling the framework HOW,
you declare WHAT you want.

Example:

```
@FeignClient(name = "product-service")  
public interface ProductClient {  
  
    @GetMapping("/products/{id}")  
    Product getProduct(@PathVariable Long id);  
}
```

```
}
```

Spring automatically generates the HTTP client.

You never write HTTP code.

Characteristics

- ✓ Less code
 - ✓ Higher productivity
 - ✓ Framework handles implementation
-

Reactive Programming

Instead of processing objects,

you process streams.

`Mono<Product>`

or

`Flux<Product>`

Nothing executes immediately.

Execution starts only when someone subscribes.

Programming Style Comparison

Style	Description	Example
Imperative	Tell the computer HOW	RestTemplate
Fluent API	Chain methods	RestClient, WebClient
Declarative	Declare WHAT you need	OpenFeign
Reactive	Process data streams	WebClient

6. CompletableFuture

Before Reactive Programming became popular,

Java introduced **CompletableFuture**.

It allows asynchronous programming without blocking the calling thread.

Traditional Execution

Order Service

↓

Call Inventory

↓

Wait

↓

Continue

CompletableFuture

Order Service

↓

Start Inventory Request

↓

Continue Other Work

↓

Inventory Completes

↓

Process Result

Example

```
CompletableFuture<Product> future =  
    CompletableFuture.supplyAsync(() ->  
        productService.getProduct(id));
```

Continue doing other work...

Later:

```
Product product = future.join();
```

Characteristics

- ✓ Standard Java
 - ✓ Asynchronous
 - ✓ Parallel execution
 - ✓ Good for Spring MVC
-

When should we use CompletableFuture?

Good for:

- Parallel tasks
- CPU-intensive work
- Spring MVC applications

Not recommended inside WebFlux applications.

CompletableFuture vs Blocking

Blocking

Request

↓

Wait

↓

Continue

CompletableFuture

Request

↓

Continue immediately

↓

Future completes

↓

Use result

CompletableFuture vs Reactive

CompletableFuture	Reactive
Java Standard Library	Project Reactor
Async	Async + Streams
One result	Mono or Flux
Thread-based	Event Loop

Reactive is much more powerful.

7. Reactive Programming

Reactive Programming is an asynchronous programming model based on streams.

Instead of returning data immediately,

we return a stream that will eventually produce data.

Example:

Mono<Product>

instead of

Product

Reactive programming focuses on:

- Non-Blocking execution
 - High scalability
 - Backpressure
 - Event-driven processing
-

Why Reactive?

Imagine 100,000 clients connected simultaneously.

Blocking servers require thousands of threads.

Reactive servers require only a small thread pool.

Traditional MVC

Request

↓

Thread

↓

Response

↓

Thread released

Reactive WebFlux

Request

↓

Event Loop

↓

Continue processing

↓

Response ready

Much higher scalability.

8. Mono and Flux

Reactive programming introduces two fundamental types.

Mono

Represents **zero or one** item.

`Mono<Product>`

Example:

```
Mono<Product> product =  
    productRepository.findById(1L);
```

Think of it as:

`Optional<Product>`

but asynchronous.

Flux

Represents **zero to many** items.

Flux<Product>

Example:

```
Flux<Product> products =  
    productRepository.findAll();
```

Equivalent to

List<Product>

but asynchronous.

Mono vs Flux

Type	Items
------	-------

Mono	0..1
------	------

Flux	0..N
------	------

Business Examples

Get one product

↓

Mono<Product>

Get all products

↓

Flux<Product>

Receive live stock updates

↓

Flux<Stock>

9. Eventual Consistency

Traditional systems expect immediate consistency.

Microservices often accept a small delay.

Example

Customer places an order.

Immediately

Order Created

One second later

Inventory Updated

Two seconds later

Email Sent

Three seconds later

Analytics Updated

Eventually,

all services become consistent.

This is called:

Eventual Consistency

Why?

It improves:

- Scalability

- Performance
- Availability

at the cost of immediate consistency.

Real Example

Amazon does not wait for:

- Email
- Analytics
- Recommendations

before confirming your order.

The confirmation appears immediately.

Everything else happens later.

Key Takeaways

- Microservices communicate through **Synchronous** or **Asynchronous** communication.
- Always decide the **communication model first**, then choose the technology.
- **Blocking** and **Non-Blocking** describe how threads execute; they are different from Sync and Async.
- Spring supports four programming styles:
 - **Imperative**
 - **Fluent API**
 - **Declarative**
 - **Reactive**
- **CompletableFuture** provides asynchronous programming for traditional Java applications.
- **Reactive Programming** builds on asynchronous, non-blocking streams using **Mono** and **Flux**.
- **Mono** represents **0 or 1** item, while **Flux** represents **0 to many** items.
- **Eventual Consistency** is a fundamental principle in distributed systems, allowing different services to become consistent over time rather than immediately.

Part 2 — Communication Patterns, Messaging, Event Streaming & Distributed Systems

8. Asynchronous Communication

What is Asynchronous Communication?

The caller sends a message and continues working immediately.

It does **NOT wait** for a response.

Order Service

|

| Publish Event

v

Kafka

|

+-----+

|

v

Inventory Service

|

v

Notification Service

The sender and receiver are decoupled.

Benefits

Better Scalability

Services are not waiting.

Better Resilience

Consumer may be temporarily offline.

Loose Coupling

Services know only the event.

High Throughput

Handles huge traffic spikes.

Drawbacks

More Complexity

Events, retries, dead letters.

Eventual Consistency

Data becomes consistent later.

Harder Debugging

No direct request-response flow.

9. Eventual Consistency

Traditional Consistency

Order Created
Stock Updated
Payment Charged
Email Sent

All completed immediately

Eventual Consistency

Order Created

Immediately:
Order Database Updated

Later:
Inventory Updated
Payment Processed
Email Sent

Analytics Updated

System becomes consistent over time.

Example

Customer places order.

T0 → Order Created

T1 → OrderPlaced Event Published

T2 → Inventory Updated

T3 → Email Sent

T4 → Analytics Updated

Everything eventually reaches the correct state.

When Acceptable?

Good

Emails

Notifications

Analytics

Audit logs

Recommendations

Not Good

Bank transfer confirmation

Stock validation before purchase

Authentication

10. RabbitMQ

What is RabbitMQ?

RabbitMQ is a Message Broker.

It delivers messages between services.

Producer

|
v
RabbitMQ
|
Consumer

Messaging Model

Producer
|
Exchange
|
Queue
|
Consumer

Characteristics

Reliable Delivery

Messages stay in queue.

Routing

Can route messages using rules.

Acknowledgements

Consumer confirms processing.

Great for Business Workflows

Example

Order Service
|
OrderCreated
|
RabbitMQ Queue
|
Email Service

Send email later.

Best Use Cases

Email Sending
SMS
Background Jobs
Payment Processing
Task Scheduling
Work Queues

11. Kafka

What is Kafka?

Kafka is an Event Streaming Platform.

Designed for massive throughput.

Producer
|
Topic
|
Consumer Group

Kafka Topic

orders-topic

Contains ordered stream of events.

Order 1
Order 2
Order 3
Order 4

Multiple Consumers

Inventory
^
|
Orders Topic -----+
|

v
Notification

|
v
Analytics

One event serves many systems.

Characteristics

Extremely Fast

Millions of events.

Durable

Events stored on disk.

Replayable

Can replay old events.

Distributed

Built for scale.

Best Use Cases

Event Sourcing
Analytics
Streaming
Microservices Events
Audit Logs
Large Platforms

12. RabbitMQ vs Kafka

Feature	RabbitMQ	Kafka
---------	----------	-------

Type	Message Broker	Event Streaming Platform
Ordering	Queue Order	Partition Order
Throughput	Medium	Extremely High
Replay Events	No	Yes
Retain History	Usually No	Yes
Best For	Tasks	Events
Complexity	Lower	Higher
Learning Curve	Easier	Harder

Rule of Thumb

RabbitMQ

Do this work

Task-oriented.

Kafka

This event happened

Event-oriented.

Example

RabbitMQ

Send Email

Generate PDF

Resize Image

Worker executes task.

Kafka

OrderPlaced

PaymentCompleted
UserRegistered

Multiple services react.

13. Spring Cloud Bus

What Problem Does It Solve?

Imagine:

Config Server

```
|
+---- Product Service
|
+---- Order Service
|
+---- Gateway
```

Config changes.

How do all services refresh?

Without Bus:

POST /actuator/refresh

Product
Order
Gateway

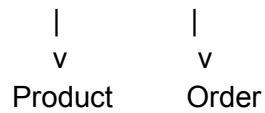
One by one

Painful.

With Spring Cloud Bus

Config Server

```
|
v
RabbitMQ/Kafka
|
+-----+
```



One refresh triggers all services.

Flow

Git Update



Config Server



Bus Event



All Services Refresh

Automatic.

Uses RabbitMQ or Kafka

spring-cloud-starter-bus-amqp

RabbitMQ

or

spring-cloud-starter-bus-kafka

Kafka

14. gRPC

What is gRPC?

High-performance communication protocol created by Google.

Uses:

HTTP/2
Protocol Buffers

instead of JSON.

REST

GET /products/1

Response:

```
{  
  "id":1,  
  "name":"Laptop"  
}
```

gRPC

Proto Contract:

```
service ProductService {  
  rpc GetProduct(ProductRequest)  
    returns (ProductResponse);  
}
```

Advantages

Faster

Binary protocol.

Smaller Payloads

Less network traffic.

Strong Contracts

Compile-time safety.

Streaming Support

Built-in.

Disadvantages

Harder Debugging

Not human-readable.

Browser Support

Less friendly than REST.

When to Use gRPC?

Internal Microservices

Inventory ↔ Product

Order ↔ Payment

High-performance communication.

NOT Ideal

Public APIs

Mobile Clients

Frontend Apps

REST usually easier.

15. CompletableFuture

Why?

Sometimes synchronous code blocks.

We want concurrency without full reactive programming.

Traditional

```
Product product =  
    productClient.getProduct(id);
```

```
Inventory inventory =  
    inventoryClient.getInventory(id);
```

Sequential.

500ms + 500ms = 1000ms

CompletableFuture

```
CompletableFuture<Product> productFuture =  
    CompletableFuture.supplyAsync(  
        () -> productClient.getProduct(id));
```

```
CompletableFuture<Inventory> inventoryFuture =  
    CompletableFuture.supplyAsync(  
        () -> inventoryClient.getInventory(id));
```

Parallel execution.

max(500ms,500ms)
≈ 500ms

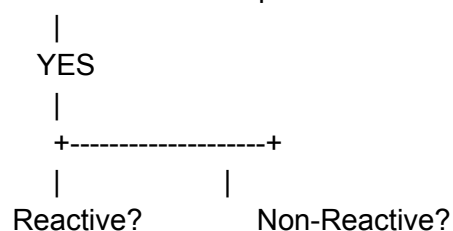
When to Use?

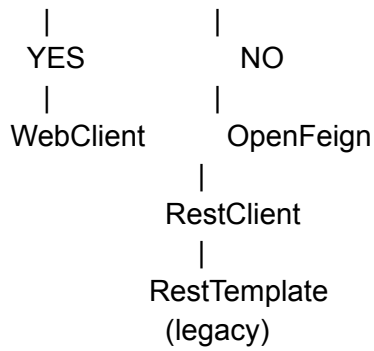
Good middle ground.

Not Reactive
Need Parallel Calls
Already Using MVC

16. Final Decision Tree

Need immediate response?





Need immediate response?

NO

Event Driven?

YES

Kafka

Need reliable task processing?

YES

RabbitMQ

Need configuration refresh?

YES

Spring Cloud Bus

Need maximum performance
between internal services?

YES

gRPC

17. What We Use In This Course

Session 6

OpenFeign

Service-to-Service Sync Communication.

Session 7

Kafka

Asynchronous Event Communication.

Session 10+

CompletableFuture

Parallel Processing.

Session 12+

Saga Pattern

Kafka

Distributed Transactions

Session 15+

Reactive WebClient

Advanced Reactive Communication.

Session 20+

Spring Cloud Bus

Distributed Configuration Refresh.

Key Takeaways

RestTemplate

Old

Blocking
Deprecated

RestClient

Modern synchronous client
Recommended MVC replacement

OpenFeign

Best synchronous service-to-service communication

WebClient

Reactive
Non-blocking
High scalability

CompletableFuture

Parallel execution without Reactive

RabbitMQ

Task processing
Work queues
Reliable delivery

Kafka

Event streaming
High throughput
Event-driven architecture

Spring Cloud Bus

Refresh configuration across all services

gRPC

Fastest service-to-service communication
Binary protocol
Internal systems

Golden Rule

Need answer NOW?

→ OpenFeign / RestClient / WebClient

Need answer LATER?

→ Kafka

Need a TASK executed?

→ RabbitMQ

Need Config Refresh?

→ Spring Cloud Bus

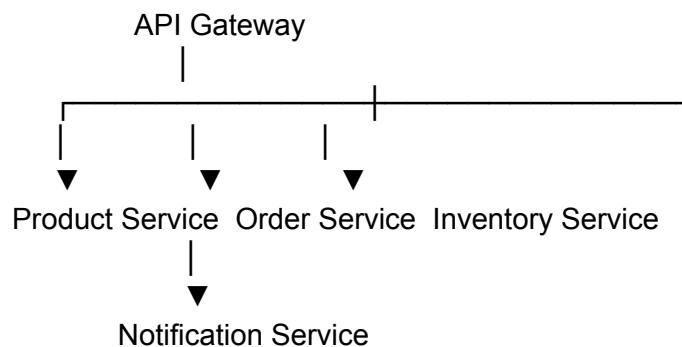
Need Maximum Performance?

→ gRPC

Part 3 — Building Synchronous Communication in Spring Boot

18. Example Platform

Throughout this course we use one business platform.



Each service owns its own database.

Product DB

Order DB

Inventory DB

Notification DB

No service is allowed to access another service's database.

Only APIs or Events.

Example Business Scenario

Customer clicks:

Place Order

Order Service must:

1. Check Inventory

2. Create Order
3. Send Email

Question:

Should every step be Sync?

No.

Inventory check:

Need answer immediately.

↓

Sync

Email notification:

Can happen later.

↓

Async

19. Example 1 — Check Inventory

Customer places an order.

Order Service asks Inventory Service.

Order Service

| GET /inventory/15



Inventory Service

|



Stock = 25

Order Service waits.

If stock exists

↓

Continue.

This is synchronous communication.

Spring Boot Flow

Controller

↓

Service

↓

Feign Client

↓

HTTP

↓

Inventory Controller

Example

OrderController

↓

OrderService

↓

InventoryClient

↓

Inventory Service

20. Example 2 — Send Email

Customer places order.

Do we really need to wait until email is sent?

No.

Instead:

Order Created

↓

Publish Event

↓

Kafka

↓

Notification Service

↓

Send Email

Customer immediately receives:

Order Created Successfully

Email arrives later.

Why?

Because email is not required to continue.

21. OpenFeign Example

Suppose Inventory Service exposes

GET /api/inventory/{productId}

Returns

```
{  
  "productId":1,  
  "available":true,  
  "quantity":20  
}
```

Instead of writing HTTP code manually...

We simply define an interface.

```
@FeignClient(name = "inventory-service")  
public interface InventoryClient {  
  
    @GetMapping("/api/inventory/{id}")  
    InventoryResponse getInventory(  
        @PathVariable Long id);  
  
}
```

Done.

Spring generates everything.

Then simply call it.

```
InventoryResponse inventory =  
    inventoryClient.getInventory(productId);
```

Looks like a normal Java method.

But internally it performs HTTP.

What Happens Internally?

```
inventoryClient.getInventory(1)
```

↓

Feign Proxy

↓

HTTP Request

↓

Inventory Service

↓

JSON Response

↓

Java Object

↓

Return

Everything is automatic.

22. RestClient Example

Spring Boot 3.2 introduced RestClient.

Instead of interfaces...

You build requests fluently.

```
InventoryResponse response =  
    restClient.get()  
        .uri("/api/inventory/{id}", id)  
        .retrieve()  
        .body(InventoryResponse.class);
```

Cleaner than RestTemplate.

Still synchronous.

23. WebClient Example

Reactive version.

```
Mono<InventoryResponse> response =  
    webClient.get()  
        .uri("/api/inventory/{id}", id)  
        .retrieve()  
        .bodyToMono(InventoryResponse.class);
```

Notice:

Returns

`Mono<InventoryResponse>`

Not

`InventoryResponse`

Later

```
response.subscribe(...)
```

or

```
return response;
```

Reactive applications rarely block.

24. RestTemplate Example

Old approach.

```
InventoryResponse response =  
    restTemplate.getForObject(  
        "/api/inventory/{id}",  
        InventoryResponse.class,  
        id  
    );
```

Still works.

But deprecated.

Not recommended for new projects.

25. Comparing the Code

RestTemplate

```
InventoryResponse response =  
restTemplate.getForObject(...);
```

Old

Blocking

Verbose

RestClient

```
restClient.get()  
    .uri(...)  
    .retrieve()  
    .body(...)
```

Modern

Blocking

Recommended for MVC

OpenFeign

```
inventoryClient.getInventory(id);
```

Simplest

Declarative

Excellent for Microservices

WebClient

```
webClient.get()
    .retrieve()
    .bodyToMono(...)
```

Reactive

Non-blocking

Highest scalability

26. Which One Is Easier?

RestTemplate

★★☆☆☆

Need manual coding.

RestClient

★★★★☆

Very readable.

OpenFeign

★★★★★

Almost zero HTTP code.

WebClient

★★★★☆

Requires understanding Reactive programming.

27. Example Decision

Scenario:

Order Service needs stock level.

Need answer immediately?

YES

↓

OpenFeign

Scenario:

Gateway calls Product Service.

Reactive Gateway.

↓

WebClient

Scenario:

Legacy MVC Application.

↓

RestClient

Scenario:

Old Spring Boot project.

↓

RestTemplate

(No new development.)

28. Why Doesn't Feign Support Reactive?

Feign is built around

HTTP Request

↓

Wait

↓

Response

Blocking model.

Reactive applications never wait.

Therefore

Reactive applications prefer

WebClient

29. WebClient Reactive Flow

Traditional

Client

↓

Wait

↓

Server

↓

Response

↓

Continue

Reactive

Client

↓

Request Sent

↓

Continue Working

↓

Server Responds Later

↓

Callback Executes

No waiting.

30. Example Timeline

Blocking

Time

0 ms

↓

Call Inventory

↓

Wait

↓

500 ms

↓

Continue

Reactive

Time

0 ms

↓

Send Request

↓

Continue Other Work

↓

500 ms

↓

Inventory Response Arrives

CPU is free during waiting.

31. What Will We Build?

Session 6

Order Service

↓

OpenFeign

↓

Inventory Service

Real synchronous communication.

Session 7

Order Service



Kafka



Notification Service

Real asynchronous communication.

Session 15

API Gateway



WebClient



Reactive Services

Advanced reactive communication.

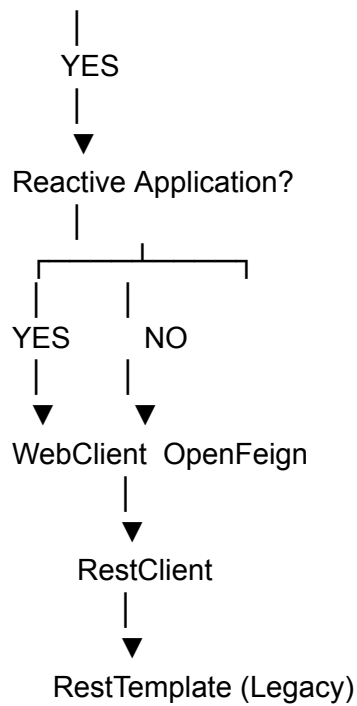
Key Takeaways

- ✓ Every Microservice owns its own database.
- ✓ Services communicate using APIs or Events.
- ✓ OpenFeign is the easiest solution for synchronous service-to-service communication.
- ✓ RestClient is the modern replacement for RestTemplate in Spring MVC applications.
- ✓ WebClient is designed for Reactive applications.
- ✓ RestTemplate is deprecated and should only appear in legacy projects.

✓ Choose the communication style based on the business requirement—not personal preference.

Final Decision Flow

Need immediate response?



Need response later?



Need reliable task queue?



Need distributed config refresh?



Spring Cloud Bus

Need ultra-fast internal communication?



gRPC

.